



DEVELOPER QUEST

AI-Assisted Full Stack Developer

AI를 활용해 아이디어를 현실의 서비스로 만드는 개발자
현장에서 얻은 경험을 바탕으로 실제 업무의 문제를 서비스로 구현합니다. AI를 개발 도구로
활용하여 아이디어를 빠르게 검증하고, 프론트엔드, 백엔드, 데이터베이스, Docker, AWS
배포까지 직접 경험했습니다.

PROJECT DUNGEON

실제 경험을 서비스로 구현한 프로젝트

Project



Tannus MES



Tannus MES 실제 제조·물류 프로세스를 디지털화한 MES 시스템

[Excel](#)[README ↗](#)[DEMO ▶](#)

Demo Account

관리자

ID : admin

PW : 1234

포장팀

ID : pack

PW : 1234

물류팀

ID : logi

PW : 1234

기술 스택

Java

Spring Boot

JSP

MyBatis

MySQL

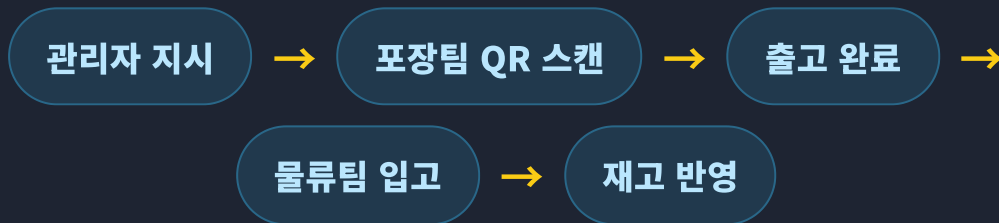
Docker

AWS

주요 기능

- 관리자 / 물류팀 / 포장팀 권한 분리
- 포장 지시 → QR 스캔 → 출고 완료 → 입고 확정 흐름
- 재고 조회와 재고 히스토리 기록
- 수정 요청 승인 / 반려 프로세스
- AWS EC2와 Docker 기반 배포

서비스 흐름도



핵심 구현 코드

QR 상태값 검증

```
if (status.equals("READY_TO_SHIP")) {  
    packingService.scanQr(qrId);  
}
```

부분 스캔 처리

```
scanQty++;  
  
if (scanQty >= orderQty) {  
    packingMapper.updateStatus(  
        packingId,  
        "READY_TO_SHIP"  
    );  
}
```

입고 확정 및 재고 반영

```
@Transactional
public void confirmReceive(...) {

    stockMapper.increaseStock(
        itemId,
        receiveQty
    );

    stockHistoryMapper.insert(
        itemId,
        "IN",
        receiveQty
    );
}
```

권한(Role) 검증

```
if (!loginRole.equals("PACKING")) {
    throw new AccessDeniedException(
        "권한이 없습니다."
    );
}
```

API / 자동화 연동

QR 스캔 처리

html5-qrcode를 활용해 현장에서 QR을 스캔하고 상태값을 변경했습니다.

```
Html5QrcodeScanner scanner =  
new Html5QrcodeScanner("reader", config);  
  
scanner.render(onScanSuccess);
```

재고 히스토리 기록

입고/출고/조정 이력을 DB에 저장하여 재고 변동을 추적했습니다.

```
stockHistoryMapper.insert(  
    itemId,  
    "IN",  
    qty,  
    loginUser  
);
```

Docker / AWS 배포

Spring Boot 프로젝트는 jar 빌드 후 Docker 컨테이너로 실행했고, React 프로젝트는 Vite 빌드 후 정적 파일을 서비스했습니다. AWS EC2 보안그룹에서 프로젝트별 포트를 열고 실제 외부 접속까지 확인했습니다.

```
docker ps
docker logs container-name --tail=100
docker compose up -d --build
```

트러블슈팅

역할별 화면 분리

문제

관리자, 물류팀, 포장팀이 같은 화면을 보면 업무 흐름이 섞이는 문제가 있었습니다.

해결

세션의 role 값을 기준으로 메뉴, 버튼, 접근 가능 화면을 분리했습니다.

QR 상태값 관리

문제

이미 처리된 QR을 다시 스캔할 때 상태값이 꼬일 수 있었습니다.

해결

상태별 허용 동작을 분리하고, 이미 완료된 작업은 재처리되지 않도록 막았습니다.

재고 이력 추적

문제

현재 재고만 보면 입고/출고가 언제 왜 변경됐는지 추적하기 어려웠습니다.

해결

IN / OUT / ADJUST 타입으로 재고 히스토리를 기록해 변경 이력을 확인할 수 있게 했습니다.

무발주 입고 처리

문제

현장에서는 발주 없이 입고되는 예외 상황이 있어 기존 포장/입고 흐름만으로 처리하기 어려웠습니다.

해결

무발주 요청 상태를 따로 두고 승인 후 입고 확정 시 재고에 반영되도록 분리했습니다.

GAME CLEAR

THINK LOAD

사용자의 문제를 이해하고
기술로 해결하는 개발자가 되겠습니다.



CONTACT

 zzangmait1524@gmail.com

 010-7210-1527

 github.com/BowWowBow

 Gimhae, Republic of Korea

실제 경험을 서비스로 구현한 프로젝트

Project



AutoOrder

 AutoOrder 재고를 분석하여 자동으로 발주를 생성하는 시스템

[📖 README ↗](#)[🎥 DEMO ▶](#)

Demo Account

관리자

ID : admin

PW : 1234

직원

ID : staff

PW : 1111

기술 스택

Java

Spring Boot

JSP

MyBatis

MySQL

Scheduler

Docker

AWS

주요 기능

- 현재고와 최소재고 비교
- 업체별 발주 요일 관리
- 추천 발주 수량 자동 계산
- 입고 완료 시 재고 자동 반영
- 발주 상태 WAITING / CONFIRMED 관리

서비스 흐름도

상품 재고 확인



최소재고 비교



발주 수량 계산



발주 생성



입고 후 재고 반영

핵심 구현 코드

박스 단위 발주 계산

```
int shortage = minQty - currentQty;

int orderQty =
    (int) Math.ceil(
        (double) shortage / unit
    ) * unit;
```

중복 발주 방지

```
int count =
    orderMapper.countTodayOrder(productId);

if (count == 0) {

    orderMapper.insertAutoOrder(productId);

}
```

API / 자동화 연동

자동발주 스케줄러

매일 정해진 시간에 업체 발주일과 재고 조건을 확인해 자동발주를 생성했습니다.

```
@Scheduled(cron = "0 0 9 * * *")
public void createAutoOrders() {
    autoOrderService.createTodayOrders();
}
```

입고 후 재고 자동 반영

발주 확정 후 입고 처리 시 상품 재고를 자동으로 증가시켰습니다.

```
product.setCurrentQty(
    product.getCurrentQty() + receivedQty
);
```

Docker / AWS 배포

Spring Boot 프로젝트는 jar 빌드 후 Docker 컨테이너로 실행했고, React 프로젝트는 Vite 빌드 후 정적 파일을 서비스했습니다. AWS EC2 보안그룹에서 프로젝트별 포트를 열고 실제 외부 접속까지 확인했습니다.

```
docker ps
docker logs container-name --tail=100
docker compose up -d --build
```

트러블슈팅

중복 발주 방지

문제

같은 상품이 같은 날짜에 여러 번 자동발주될 수 있었습니다.

해결

상품 ID와 날짜 기준으로 중복 여부를 체크한 뒤 발주를 생성했습니다.

발주 단위 계산

문제

부족 수량 그대로 발주하면 박스 단위와 맞지 않았습니다.

해결

부족 수량을 발주 단위로 올림 계산하여 실제 업무에 맞게 처리했습니다.

스케줄러 시간 검증

문제

자동발주가 실행되지 않을 때 서버 시간과 실행 로그 확인이 필요했습니다.

해결

서버 시간, 스케줄러 로그, DB 반영 결과를 함께 확인했습니다.

업체별 발주 요일 관리

문제

모든 상품을 매일 발주하면 실제 업체별 납품 주기와 맞지 않았습니다.

해결

업체별 발주 요일과 입고 요일을 기준으로 오늘 발주 대상만 자동 생성되도록 처리했습니다.

GAME CLEAR

THINK LOAD

사용자의 문제를 이해하고
기술로 해결하는 개발자가 되겠습니다.



CONTACT

 zzangmait1524@gmail.com



010-7210-1527



github.com/BowWowBow



Gimhae, Republic of Korea

실제 경험을 서비스로 구현한 프로젝트

Project



ToyShop

 ToyShop 회원부터 결제까지 구현한 쇼핑몰 프로젝트

[📖 README ↗](#)[🎥 DEMO ▶](#)

Demo Account

관리자

ID : admin

PW : 1234

일반회원

ID : staff1

PW : 1234

일반회원

ID : staff2

PW : 1234

일반회원

ID : staff3

PW : 1234

✅ 회원가입도 가능합니다.

기술 스택

Java

Spring Boot

Thymeleaf

MyBatis

MySQL

Toss Payments

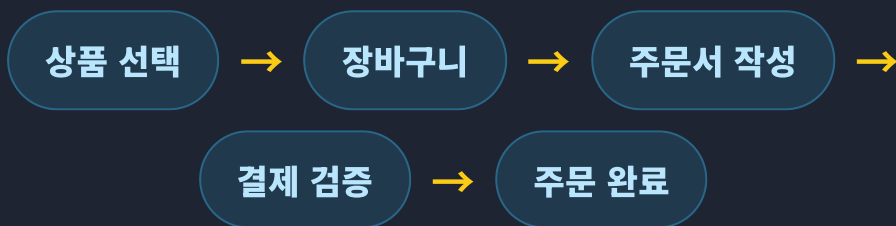
Docker

AWS

주요 기능

- 회원가입 / 로그인 / 회원정보 수정
- 상품 목록 / 상세 / 검색
- 장바구니 / 바로구매 / 주문
- 무통장 입금과 Toss Payments 테스트 결제
- 관리자 상품 / 주문 / 회원 관리

서비스 흐름도



핵심 구현 코드

결제 금액 검증

```
if (orderAmount != paidAmount) {  
    throw new IllegalStateException("결제 금액 불일치");  
}
```

주문 상태 관리

```
order.setOrderStatus("ORDER_COMPLETE");  
order.setPaymentStatus("PAID");
```

API / 자동화 연동

Toss Payments 결제

Toss Payments 테스트 결제를 연동하고 결제 금액 검증을 처리했습니다.

```
confirmPayment(  
    paymentKey,  
    orderId,  
    amount  
);
```

Naver SMTP 이메일

아이디/비밀번호 찾기 기능에서 이메일 인증 흐름을 구현했습니다.

```
mailSender.send(message);
```

Docker / AWS 배포

Spring Boot 프로젝트는 jar 빌드 후 Docker 컨테이너로 실행했고, React 프로젝트는 Vite 빌드 후 정적 파일을 서비스했습니다. AWS EC2 보안그룹에서 프로젝트별 포트를 열고 실제 외부 접속까지 확인했습니다.

```
docker ps  
docker logs container-name --tail=100  
docker compose up -d --build
```

트러블슈팅

결제 검증

문제

결제 성공 후 실제 주문 금액과 결제 금액 검증이 필요했습니다.

해결

paymentKey, orderId, amount 값을 기준으로 결제 승인 로직을 구성했습니다.

재고 차감 타이밍

문제

주문 생성 시점과 결제 완료 시점 중 언제 재고를 차감할지 기준이 필요했습니다.

해결

결제 방식과 주문 상태를 분리하여 재고 반영 시점을 명확히 했습니다.

이메일 인증 흐름

문제

아이디/비밀번호 찾기에서 인증 메일 전송과 검증 흐름이 필요했습니다.

해결

Naver SMTP를 연결하고 인증 코드 생성, 메일 발송, 사용자 검증 단계를 분리했습니다.

주문 상태 되돌리기

문제

관리자에서 결제확인, 상품준비, 배송중 상태를 잘못 변경했을 때 복구할 기준이 필요했습니다.

해결

주문 상태와 배송 정보를 분리하고 단계별 되돌리기 로직을 만들어 이전 상태로 복구할 수 있게 했습니다.

대표 이미지 예외 처리

문제

상품 이미지가 없거나 잘못된 경로일 경우 깨진 이미지가 표시되어 화면 품질이 떨어지는 문제가 있었습니다.

해결

기본 이미지(no-image.png)를 제공하고 이미지가 없으면 자동으로 대체 이미지를 출력하도록 처리했습니다.

기본 배송지 관리

문제

배송지를 여러 개 등록한 회원은 주문할 때마다 배송지를 다시 선택해야 하는 불편함이 있었습니다.

해결

기본 배송지 기능을 추가하여 주문 페이지 진입 시 기본 배송지가 자동으로 선택되도록 구현했습니다.

GAME CLEAR

THINK LOAD

사용자의 문제를 이해하고
기술로 해결하는 개발자가 되겠습니다.



CONTACT

✉ zzangmait1524@gmail.com



010-7210-1527



github.com/BowWowBow



Gimhae, Republic of Korea

실제 경험을 서비스로 구현한 프로젝트

Project



WhisperMe



WhisperMe OpenAI 기반 AI 상담 및 생산성 웹서비스

 **README** ↗

 **DEMO** ▶



Demo Account

일반회원

ID : test

PW : 1234

일반회원

ID : test1

PW : 1234



회원가입도 가능합니다.

기술 스택

React

Spring Boot

MyBatis

MySQL

OpenAI API

Docker

AWS

주요 기능

- AI 채팅과 대화방 관리
- 이미지 업로드 및 AI 분석
- 오늘 할 일 / 미래 일정 / 지난 내역 관리
- 날씨와 운세 기능
- React 프론트와 Spring Boot 백엔드 분리 배포

서비스 흐름도

채팅방 생성



메시지 입력



AI 요청



답변 저장



대화 기록 조회

핵심 구현 코드

OpenAI 대화 요청

```
String answer = openAiService.askWithHistory(messages);  
  
chatMapper.insertAssistantMessage(roomId, answer);
```

이미지 분석 요청

```
String result = openAiService.askWithImage(  
    prompt,  
    imageBase64  
);
```

API / 자동화 연동

OpenAI API

OpenAI API를 활용해 텍스트 대화와 이미지 분석 답변을 생성했습니다.

```
openAiService.askWithHistory(messages);
```

날씨 / 위치 API

브라우저 위치 정보를 기반으로 날씨 정보를 불러와 사용자에게 표시했습니다.

```
navigator.geolocation.getCurrentPosition(  
    successCallback,  
    errorCallback  
);
```

음성 인식

Web Speech API를 활용해 한국어 음성 입력 기능을 구현했습니다.

```
recognition.lang = "ko-KR";  
recognition.start();
```

네이버 뉴스 검색 API

최신 정보가 필요한 질문은 Naver Search API로 검색 후 AI 답변에 반영했습니다.

```
if (questionRouter.needSearch(question)) {  
    List<NewsDto> news =  
        naverSearchService.search(keyword);  
  
    return aiService.answerWithNews(question, news);  
}
```

Docker / AWS 배포

Spring Boot 프로젝트는 jar 빌드 후 Docker 컨테이너로 실행했고, React 프로젝트는 Vite 빌드 후 정적 파일을 서비스했습니다. AWS EC2 보안그룹에서 프로젝트별 포트를 열고 실제 외부 접속까지 확인했습니다.

```
docker ps
docker logs container-name --tail=100
docker compose up -d --build
```

트러블슈팅

localhost API 문제

문제

AWS 배포 후 프론트에서 localhost로 API를 호출해 백엔드 연결이 실패했습니다.

해결

API 주소를 EC2 퍼블릭 IP 기준으로 변경하여 외부 브라우저에서도 정상 호출되게 했습니다.

모바일 채팅창 깨짐

문제

모바일에서 100vh와 fixed 입력창 때문에 채팅 입력창이 잘렸습니다.

해결

100dvh와 flex column 구조를 적용하고 safe-area를 고려했습니다.

이미지 분석 요청 처리

문제

이미지와 텍스트를 함께 보낼 때 AI 요청 형식이 맞지 않아 분석이 실패했습니다.

해결

이미지를 Base64로 변환하고 텍스트 프롬프트와 함께 요청하도록 구조를 분리했습니다.

대화방 자동 선택

문제

새 채팅방을 만든 뒤 바로 선택되지 않아 사용자가 다시 방을 클릭해야 했습니다.

해결

채팅방 생성 응답을 받은 후 `selectedRoom`을 즉시 갱신하고 메시지 입력 화면으로 연결했습니다.

GAME CLEAR

THINK LOAD

사용자의 문제를 이해하고
기술로 해결하는 개발자가 되겠습니다.



CONTACT

✉ zzangmait1524@gmail.com



010-7210-1527



github.com/BowWowBow



Gimhae, Republic of Korea